

APUNTESWEB · INGENIERÍA INFORMÁTICA

DESOFT

Segundo curso · 2.º cuatrimestre · 10 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción a UML

UML es un lenguaje gráfico para visualizar, especificar, construir y documentar. Proporciona una forma estándar de describir los planos de un sistema.

Un **modelo** es una simplificación de la realidad, completa y consistente. Los buenos modelos resaltan elementos de influencia y oculta los no relevantes para el **nivel de abstracción** dado. Pueden ser **estructurales** o de **comportamiento**.

Se modela para producir software de calidad duradera que satisfaga de forma consistente y predecible necesidades cambiantes, cumpla su propósito y sea rápido y efectivo. Hay dos **enfoques** diferentes, el **algorítmico** (tradicional) y el **orientado a objetos** (moderno).

Principios del modelado:

- La elección de los **modelos** influye en como se acomete un problema y su solución
- Todo modelo se puede expresar en distintos niveles de **abstracción**.
- Los mejores modelos están ligados a la **realidad**.
- No es suficiente con un único modelo.

Los bloques de construcción de UML son **elementos, relaciones y diagramas**.

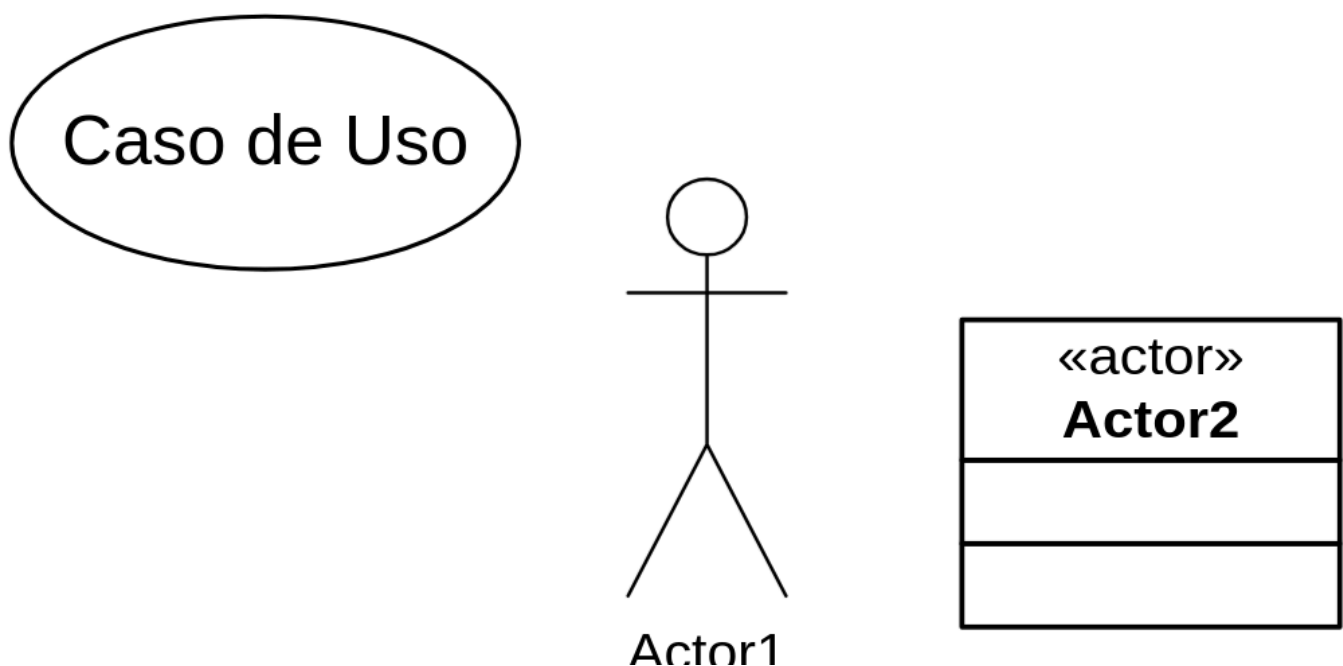
Un **diagrama** es un grafo conexo de nodos y arcos. Un diagrama bien estructurado se centra en un único aspecto del sistema, contiene solo elementos y relaciones esenciales para el, y proporciona detalles consistentes con su nivel de abstracción.

2. Modelado de Casos de Uso

(diagrama de casos de uso)

Los **casos de uso** capturan el comportamiento deseado del sistema sin especificar como implementarlo. Son conjuntos de **secuencias** de acciones que ejecuta un sistema para producir un **resultado observable** de valor para un actor.

Una **secuencia** es una interacción del sistema con **elementos externos**. Un **actor** es un **rol que juega un usuario** o un sistema al interactuar con el sistema. El mismo actor puede desenvolver varios roles. Un **escenario** es una instancia del caso de uso.



Los **casos de uso** se especifican textualmente como un flujo de eventos, incluyendo un **escenario principal** y sus **alternativas**. Capturan comportamientos sin especificar la implementación, y se realizan creando **colaboraciones** de clases y otros elementos. Un caso de uso bien estructurado denota un comportamiento **simple** e **identificable**, identifica **actores** que interactúan con él. Incorpora comportamiento común incluyendo otros casos de uso y coloca variantes en casos de uso que lo extienden, describe el flujo de eventos con los usuarios por medio de escenarios, y especifica **pre** y **post condiciones**.

Los **diagrama de casos de uso** visualizan el **comportamiento** de un sistema y se usan para modelar el **contexto** (que elementos están fuera y cuales dentro) y los **requisitos funcionales** (especificar que debería hacer) de un sistema. Se compone de:

- **Actores**
- **Casos de uso**
- **Asociaciones** entre actores y casos de uso

- **Relaciones** entre casos de uso (generalización, inclusión, extensión) o entre actores (generalización)

En la **generalización** se hereda el comportamiento de un caso de uso, pudiendo añadir o redefinir comportamiento, para modelar cosas de uso que hacen un poco más.

En la **inclusión**, un caso de uso incorpora el comportamiento de otro, para evitar repetir comportamiento común.

En la **extensión**, un caso de uso modifica el comportamiento de otro en **puntos de extensión**, para describir variaciones.

[!Examen Parcial]

3. Modelado Estructural

(diagramas de clases)

Una **clase** es una descripción de un conjunto de objetos que comparten atributos, operaciones, relaciones y semántica. Son **abstracciones** independientes del lenguaje de programación capaces de implementar **interfaces**.

Los objetos de una misma clase tienen los mismos tipos de **estado** y **comportamiento**. Las clases **colaboran** entre si para satisfacer necesidades. Hay 3 tipos de **relaciones**:

- **Dependencia:** los cambios en la planificación de un elemento pueden afectar a otro, indica utilización.
- **Generalización:** relación padre hijo.
- **Asociación:** relación estructural. Se usa para vincular clases. Además tenemos adornos:
 - **Nombre:** etiqueta que describe la relación
 - **Rol:** papel que juega cada clase relacionada
 - **Multiplicidad:** nº de instancias participantes
 - **Agregación:** es como lo de entidad débil - entidad fuerte de BDI (*asociación todo-partes*).
 - **Composición:** igual que en POO.

Son la base para diagramas de paquetes, componentes y despliegue. Pueden contener ****paquetes****.

4. Modelado de Comportamiento

(diagramas de interacción)

Una **interacción** es un comportamiento que incluye **mensajes** intercambiados por un conjunto de objetos dentro de un **contexto** para lograr un propósito.

Es un modelado dinámico de **colaboraciones** (*sociedad de objetos que proporcionan un comportamiento mayor que la suma de sus comportamientos*).

Un **mensaje** es una transmisión de información entre objetos para desencadenar una actividad. Pueden ser:

- **Llamadas**-> invocan operaciones.
- **Retornos**-> devuelven un valor
- **Señales**
- **Creación**
- **Destrucción**

Las interacciones sirven para modelar **flujo de control** en un sistema o subsistema. La implementación de una operación, o una clase o componente. Los objetos pueden ser **concretos** o **prototípicos**.

Los **enlaces** son conexiones semánticas entre objetos por las que se transmite un mensaje. Una **secuencia** es un flujo de mensajes encadenados entre diferentes objetos, se origina dentro de un **proceso** o **hilo** y continúa mientras este exista. Cada proceso redefine un **flujo de control** separado en el que los mensajes se ordenan según se suceden en el tiempo.

Los **diagramas de interacción** describen la forma en la que **colaboras** distintos objetos para producir un comportamiento. Contienen objetos, enlaces y mensajes, y muestran la secuencia dinámica de mensajes que fluyen por los enlaces. Suelen capturar el comportamiento de un solo **caso de uso**. Hay dos tipos **isomorfos**:

- **Diagramas de secuencia**: ordenaciones temporal de mensajes
- **Diagramas de colaboración**: organización estructural de objetos

Los **diagramas de secuencia** tienen una **línea de vida** que representa la existencia de un objeto y un **foco de control** que representa el tiempo durante el cual un objeto ejecuta una acción. Las **diagramas de colaboración** tienen un **enlace** y un **número de secuencia**.

Una **iteración** indica la repetición de un mensaje, y una **bifurcación** representa un mensaje cuya ejecución depende de la evaluación de una condición.

Los diagramas de interacción pueden modelar **colaboraciones** y **escenarios** dentro de una caso de uso. Los **diagramas de secuencia** modelan escenarios o visualizan iteraciones y bifurcaciones sencillas; los de **colaboración** son preferibles para iteraciones y bifurcaciones complejas.

5. Modelado Estructural Avanzado

5.1 Definición general

El **modelado estructural** en UML (Unified Modeling Language) se refiere a la representación de la **estructura estática del sistema**, es decir, **cómo están organizados sus elementos (clases, objetos, relaciones, interfaces, etc.)**.

Se diferencia del **modelado dinámico**, que se enfoca en el comportamiento (interacciones, eventos, flujos).

5.2. Niveles de abstracción

La UML permite modelar un sistema en **varios niveles de detalle**, lo cual es crucial para **diferenciar entre lo que se quiere hacer, cómo se piensa hacer y cómo se implementa finalmente**.

◆ Conceptual

- **Qué es:** representación del dominio de forma abstracta, sin pensar en software aún.
- **Ejemplo:** En una tienda, tienes conceptos como `Cliente`, `Producto`, `Pedido`.

◆ Especificación

- **Qué es:** definición de **interfaces** del sistema (cómo interactúan los componentes entre sí).
- **Ejemplo:** Sabemos que `Cliente` puede "realizarPedido()", pero no cómo lo hace.

◆ Implementación

- **Qué es:** ya vemos el detalle real en código, clases, métodos, campos.
- **Ejemplo:** `Cliente` tiene `List<Pedido>` como atributo, y un método `crearPedido()`.

5.3 Elementos del modelo estructural

Atributos

- A **nivel conceptual**: identifican propiedades relevantes. Ej: `nombre` de un `Cliente`.
- A **nivel de especificación**: decides tipos y restricciones (e.g., `nombre: String`, obligatorio).
- A **nivel de implementación**: el atributo es un **campo real en una clase**.

Operaciones

- A **nivel conceptual**: representan **responsabilidades**. Ej: `realizarPedido()`.
- A **nivel de especificación**: se definen visibilidad, parámetros, excepciones.
- También se incluyen **operaciones privadas y protegidas**, no visibles externamente.

Alcance

Define si el atributo es **de instancia** (cada objeto lo tiene) o **de clase** (compartido).

- **Ejemplo:**

```
class Usuario {  
    static int totalUsuarios; // atributo de clase (subrayado en UML)  
    String nombre;           // atributo de instancia  
}  
...
```

5.4 Asociaciones y relaciones

Asociación

Relación estructural entre clases. A distintos niveles, implica cosas distintas:

- **Conceptual**: denota **navegabilidad**: puedes ir de `Pedido` a `Cliente`.
- **Especificación**: muestra **responsabilidades** e **interfaces**.
- **Implementación**: implica una **estructura de datos** (por ejemplo, una lista).

```
```java  
class Cliente {
 List<Pedido> pedidos; // Asociación 1 a muchos
}
```

## Clase Asociación

Una **asociación** que necesita atributos propios se modela como **clase**. **Ejemplo**: Relación `Matricula` entre `Estudiante` y `Curso` con atributo `nota`.

## Composición vs Agregación

- **Agregación**: relación débil. Las partes pueden vivir sin el todo.
- **Composición**: relación fuerte. Las partes **viven y mueren** con el todo.

```
class Pedido {
 private List<LineaPedido> lineas; // Composición: LineaPedido no existe sin
```

```
Pedido
```

```
}
```

## 5.5 Mecanismos de extensibilidad

UML es extensible para adaptarse a dominios específicos:

### ◆ Estereotipos (`<< >>`)

Permiten **crear nuevos elementos** especializados.

- Ej: `<<controller>>`, `<<entity>>`, `<<boundary>>` en MVC.

### ◆ Valores etiquetados

Metadatos asociados a elementos UML. Ej: `versión = 1.0`.

### ◆ Restricciones

Condiciones que deben cumplirse (`{ordered}`, `{unique}`, `{readOnly}`).

## 5.6 Clasificadores

Un **clasificador** es un concepto general que agrupa cosas como:

- **Clases**
- **Interfaces**
- **Componentes**
- **Casos de uso**

Tienen **instancias**, **atributos**, **operaciones** y pueden participar en **relaciones**.

## 5.7 Visibilidad

Controla **quién puede acceder** a atributos u operaciones:

Símbolo	Nivel	Significado
<code>+</code>	Pública	Accesible desde cualquier clase
<code>-</code>	Privada	Solo accesible dentro de la clase
<code>#</code>	Protegida	Accesible desde subclases
<code>~</code>	De paquete	Accesible solo dentro del mismo paquete

```
class Cliente {
 private String nombre; // (-)
 protected void validar(); // (#)
 public void comprar(); // (+)
}
```

## 5.8 Interfaces y realización

### ◆ ¿Qué es una interfaz?

Una **colección de operaciones** sin implementación ni atributos.

- No define estructura.
- Define **qué se debe hacer**, no **cómo**.
- Puede participar en:
  - **Generalización**
  - **Dependencia**
  - **Asociación**
  - **Realización**

```
interface Imprimible {
 void imprimir();
}

class Factura implements Imprimible {
 public void imprimir() {
 // implementación concreta
 }
}
```

📌 **Realización** es la relación entre una interfaz y una clase que **garantiza** cumplir ese contrato.

## 6. Otros Diagramas de Modelado

---

### 5.1 Diagrama de Objetos

#### ¿Qué es?

Representa una **fotografía estática** de los objetos (instancias de clases) en un momento específico del tiempo.

#### Propósito

- Mostrar cómo están relacionados los objetos en **una situación concreta**.
- Ver **configuraciones ejemplo** del sistema.

#### Detalles clave

- No muestra clases, sino **especificaciones de instancias**.
- Puede incluir objetos de clases abstractas o atributos incompletos (porque es una **especificación**, no una ejecución real).

### 6.2 Diagrama de Paquetes

#### ¿Qué es?

Representa cómo se **organiza y agrupa** un sistema en paquetes lógicos.

#### Propósito

- Agrupar clases, interfaces y otros elementos.
- Mostrar **dependencias entre paquetes**, lo que ayuda a gestionar la complejidad.

#### Detalles clave

- Los paquetes pueden contener **otros paquetes**.
- Hay un **paquete raíz** que abarca todo el sistema.
- Las dependencias indican que **algún elemento interno depende de otro** (no todos).
- Permite controlar **visibilidad y encapsulamiento**.

## Analogía

Como carpetas dentro de otras carpetas en tu computadora, organizadas por temas (funcionalidad, módulo, etc.).

# 6.3 Diagrama de Componentes

## ¿Qué es?

Representa los **componentes físicos** de un sistema de software y sus **dependencias**.

## Propósito

- Mostrar cómo el sistema está dividido en **módulos reutilizables**.
- Refleja los elementos que **pueden ser compilados, desplegados o reemplazados**.

## Detalles clave

- Cada componente tiene nombre y puede tener **estereotipos** como `<<executable>>`, `<<library>>`, `<<document>>`, etc.
- Indica relaciones entre **interfaces ofrecidas y requeridas**.

## Analogía

Como las piezas de LEGO de un sistema: cada una hace algo y puede ser reemplazada o ensamblada con otras.

# 6.4 Diagrama de Despliegue

## ¿Qué es?

Muestra la **arquitectura física** de un sistema: hardware y software durante la ejecución.

## Propósito

- Modelar la **infraestructura de ejecución**.
- Visualizar dónde y cómo se **despliegan los componentes** del sistema.

## Detalles clave

- Incluye **nodos** (máquinas físicas o virtuales).
- Muestra **componentes de software** ejecutándose en esos nodos.
- Ayuda en decisiones de **distribución, rendimiento y escalabilidad**.

## Analogía

Como un plano de red de servidores, con los programas que corren en cada uno.

# 6.5 Diagrama de Estados

## ¿Qué es?

Describe cómo **cambia el estado de un objeto** a lo largo del tiempo en respuesta a eventos.

## Propósito

- Entender el **comportamiento de un objeto** durante su ciclo de vida.
- Identificar **eventos, condiciones y transiciones**.

## Detalles clave

- Se enfoca en un solo objeto (como `Pedido`, `Semáforo`, `Conexión`).
- Puede haber **superestados** que agrupan estados similares.

## Analogía

Como los estados de una puerta: *abierta*, *cerrada*, *bloqueada* y cómo cambia según acciones (eventos) como *cerrar()*, *bloquear()*.

# 6.6 Diagrama de Actividades

## ¿Qué es?

Muestra el **flujo de trabajo o procesos** en el sistema, incluso con actividades concurrentes.

## Propósito

- Representar **procesos de negocio**, lógica de un método o **tareas paralelas**.
- Modelar decisiones, ramificaciones, sincronizaciones, ciclos.

## Detalles clave

- Hereda ideas de los **diagramas de flujo, estados y redes de Petri**.
- Se usa mucho en **procesos de negocio, tareas automatizadas o casos de uso complejos**.

## Analogía

Como una receta de cocina con pasos en secuencia y en paralelo (ej. mientras hierve el agua, vas picando los ingredientes).

---

Tipo de diagrama	Enfocado en...	Ideal para...
Objetos	Instancias en un momento específico	Mostrar configuración de objetos
Paquetes	Organización y dependencias lógicas	Modularizar el sistema
Componentes	Elementos físicos del software	Ver arquitectura modular y reutilizable
Despigue	Infraestructura de ejecución	Modelar hardware y distribución
Estados	Ciclo de vida de un objeto	Mostrar cómo responde a eventos
Actividades	Flujo de trabajo y procesos	Representar lógica compleja o concurrente

## 7. Patrones de Diseño

---

### 7.1 Patrones de Diseño

Los **patrones de diseño** son **soluciones reutilizables, probadas y eficientes** a problemas de diseño que se repiten con frecuencia en el desarrollo de software.

#### Reutilización

---

- Un diseño debe adecuarse a **requisitos actuales** y anticipar **problemas futuros**.
- Los expertos **no** afrontan cada situación partiendo de **cero**, si no que **reutilizan soluciones** útiles en el pasado.
- Los patrones resuelven problemas concretos de diseño y ayudan a lograr soluciones **flexibles y reutilizables**.

#### Ventajas

---

- Cada patrón nombre, explica y evalúa un **diseño recurrente** en sistemas.
- Permiten lograr **rápidamente** un buen diseño **reutilizando** experiencia previa.
- Ayudan a **elegir alternativas** que hacen que un sistema sea reutilizable y a evitar las que provocan el efecto contrario.
- Facilitan la **documentación** y el mantenimiento de sistemas existentes.

#### Elementos

---

Elemento	Descripción
Nombre	Descripción concisa del problema de diseño
Problema	Contexto + situación en la que usar el patrón.
Solución	Estructura de clases y objetos que resuelve el problema.
Consecuencias	Ventajas, desventajas, y efectos secundarios.

### 7.2 MVC (Model-View-Controller)

**MVC** es un patrón de diseño arquitectónico muy usado en interfaces gráficas y aplicaciones web.



Componente	Rol
<b>Modelo</b>	Contiene los datos y lógica de negocio.
<b>Vista</b>	Muestra los datos del modelo al usuario.
<b>Controlador</b>	Gestiona la entrada del usuario y actualiza el modelo o la vista.

#### Ventajas:

- Vista y modelo **desacoplados**: puedes cambiar el aspecto visual sin tocar la lógica.
- Puedes **anidar vistas** o tener múltiples representaciones del mismo modelo.
- La interacción puede modificarse sin tocar la visualización.

## 7.3 Tipos de Patrones

	De creación	Estrucutales	De comportamiento
<b>Clase</b>	<b>Factory Method</b>	Adapter	Interpreter Template Method
<b>Objeto</b>	<b>Abstract Factory</b> Builder Prototype <b>Singleton</b>	Adapter Bridge <b>Composite</b> <b>Decorator</b> <b>Facade</b> Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento <b>Observer</b> <b>State</b> Strategy Visitor

## 7.4 ¿Qué problemas resuelven?

Los patrones de diseño ayudan a la:

- Identificación de los objetos necesarios.
- Determinación del tamaño de los objetos.
- Diseñar **interfaces limpias y coherentes**.
- Reutilización.
- Diseñar pensando en **la evolución del sistema**.

## Identificación de los objetos necesarios

---

Muchos elementos de un diseño proceden del análisis. Después aparecen clases **sin equivalente en el mundo real**. Los patrones ayudan a identificar las **abstracciones menos obvias**.

## Determinación del tamaño de los objetos

---

Los objetos pueden variar en tamaño y en número. Algunos patrones de diseño describen formas concretas de **descomponer un objeto en otros más pequeños**.

## Diseñar interfaces limpias y coherentes

---

Se debe **restringir el uso de referencias** a clases concretas, los objetos deben mantener un **conocimiento mutuo limitado**. Los **patrones de creación** aseguran que el sistema se diseñe en términos de interfaces.

## Reutilización

---

- **Herencia:** se define en tiempo de **compilación**, **rompe el principio de encapsulación**. (Flecha con triángulo blanco)
- **Composición:** los enlaces entre objetos se establecen en tiempo de **ejecución**. Cualquier otro objeto puede ser **reemplazado** dinámicamente por otro del **mismo tipo**. El acceso a objetos a través de interfaces **no rompe la encapsulación**.
- Principio de DOO: **equilibra el uso de la asociación y la generalización como mecanismos complementarios**. Facilita la **herencia** facilita la construcción, pero la **composición** aporta mayor **flexibilidad**.
- **Delegación:** modo de asociación que **suple a la herencia** por la cual dos objetos tratan una **petición**, un objeto receptor delega en su **ayudante**. Facilita **combinación de comportamientos** en tiempo de ejecución.

## 8. Patrones de Creación

---

### 8.1 Patrones de creación

Los **patrones de creación** abstraen el proceso de **creación de objetos**, acotando el **conocimiento** sobre las **clases** concretas usadas. Ocultan como se **crean y enlazan** las **instancias** de las clases, de manera que **el resto del sistema** sólo **conoce los objetos a través de sus interfaces**. Hacen posible **programar para interfaces** y no para implementaciones.

### 8.2 Abstract Factory (Fábrica Abstracta)

La **abstract factory** aporta una **interfaz** para crear **familias de objetos** ocultando clases concretas. Se usa para:

- Configurar un sistema escogiendo una sola **familia de productos** entre varias.
- Si cada **familia** está diseñada para el **uso conjunto de sus productos**.
- Para proporcionar **bibliotecas de productos** de los que sólo se revelan sus interfaces.

#### Participantes:

---

- **FabricaAbstracta**: Declara una interfaz para operaciones que crean productos abstractos.
- **FabricaConcreta**: Implementa operaciones para crear productos concretos.
- **ProductoAbstracto**: Declara una interfaz del tipo producto.
- **ProductoConcreto**: Define un producto a ser creado por la fábrica correspondiente.
- **Cliente**: Usa interfaces de clases abstractas

#### Estructura

---

## Ventajas e Inconvenientes - **\*\*Consistencia\*\*** entre productos de una familia. - Fácil de **\*\*sustituir familias enteras\*\***. - Alto nivel de **\*\*abstracción y flexibilidad\*\***.

- Difícil añadir nuevos **tipos de productos**.
- Puede volverse complejo si hay muchas familias/productos.

## 8.3 Factory Method (Método Fábrica)

El **factory method** define una **interfaz** para **crear objetos** cediendo a las **subclases** la decisión sobre **qué clase instanciar**. Se usa cuando:

- Una clase no puede prever la clase de objetos que debe crear.

### Participantes:

---

- **Producto**: define la interfaz de los objetos creados
- **ProductoConcreto**: implementa la interfaz **Producto**
- **Creador**: declara el método de fabricación que devuelve un objeto **Producto**.
- **CreadorConcreto**: Redefine el método de fabricación para devolver una instancia de **ProductoConcreto**.

### Estructura

---

### Ventajas y Desventajas

---

- Código desacoplado.
- Fácil de extender.
- Fomenta el uso de interfaces.
- Puede llevar a muchas subclases si tienes muchos tipos de productos.

# 9. Patrones Estructurales

---

## 9.1 Patrón Composite

Organiza objetos en **estructura de árbol** para representar jerarquías. Permite tratar de manera **uniforme** a **individuos** y **grupos**. Se usa para:

- Representar jerarquías de objetos **parte-todo**.
- Obviar diferencias entre los **individuos** y sus **composiciones**.

### Participantes

Rol	Descripción
Componente	Define la interfaz común para todos los objetos (hojas y compuestos). Puede incluir comportamiento por defecto.
Hoja	Representa objetos simples que no contienen otros. Implementan la interfaz, pero no gestionan hijos.
Compuesto	Contiene otros componentes (hojas o compuestos). Implementa operaciones para añadir, eliminar o acceder a los hijos.
Cliente	Usa objetos a través de la interfaz del componente, sin preocuparse si es hoja o compuesto.

### Estructura

---

## Ventajas e Inconvenientes - Permite **\*\*composición recursiva\*\***, es decir, compuestos dentro de compuestos. - Simplifica el código del cliente al unificar el tratamiento de todos los elementos. - Facilita añadir nuevos tipos de componentes.

- Puede ser difícil **restringir qué tipos de componentes se pueden añadir** dentro de un compuesto.
- A veces, hay métodos en la interfaz común que no tienen sentido para todas las clases (por ejemplo, `add()` en una hoja).

## 9.2 Patrón Decorator

Asigna/retira **responsabilidades adicionales** a un objeto particular **dinámicamente**. Proporciona una **alternativa flexible** a la **herencia** para ampliar funcionalidad. Suele combinarse con **Composite**. Se

usa para:

- Otorgar o revocar responsabilidades a objetos individuales de manera dinámica y transparente.
- Cuando no es viable la extensión mediante herencia

## Participantes

Rol	Descripción
Componente	Interfaz común para objetos que pueden tener funcionalidades añadidas.
ComponenteConcreto	Implementación base que puede ser decorada.
Decorador	Clase abstracta que implementa la interfaz y contiene una referencia a un componente.
DecoradorConcreto	Añade funcionalidades específicas al componente envolviendo su comportamiento.

## Estructura

## Ventajas e Inconvenientes

- Mucho más **flexible que la herencia**: puedes combinar decoradores fácilmente.
- Evita clases "gigantes" con muchas funcionalidades opcionales.
- Puedes decorar objetos en tiempo de ejecución.
- Los decoradores **no son idénticos al objeto decorado**, lo cual puede ser confuso en ciertos contextos.
- Puede generar **múltiples objetos pequeños muy similares**, que pueden ser difíciles de rastrear y depurar.

# 10. Patrones de Comportamiento

---

## 10.1 Patrón Observer

El **cambio de estado** en un objeto (sujeto) se **notifica** a los objetos **suscritos** a él (observadores), que **sincronizan** su estado con el del sujeto. Establece una **dependencia de uno a muchos** entre objetos. Se usa cuando:

- Un cambio en un objeto requiera modificar otros objetos **sin que importe cuántos** son.
- Un objeto deba ser capaz de hacer notificaciones a otros **sin hacer suposiciones sobre quiénes** son exactamente.

### Participantes

Participante	Rol
<b>Sujeto (Subject)</b>	El objeto que tiene el estado que puede cambiar. Mantiene una lista de observadores y define métodos para añadir, eliminar y notificar observadores.
<b>Observador (Observer)</b>	Interfaz que deben implementar los objetos que quieren enterarse de los cambios. Contiene el método <code>actualizar()</code> .
<b>SujetoConcreto</b>	Implementación del sujeto, guarda el estado que interesa a los observadores y los notifica cuando cambia.
<b>ObservadorConcreto</b>	Implementación del observador. Se registra al sujeto y actualiza su estado cuando recibe una notificación.

### Estructura

---

## Ventajas e Inconvenientes - **\*\*Desacopla el sujeto de los observadores\*\***: pueden evolucionar por separado. - Permite una **\*\*arquitectura reactiva\*\***, donde los componentes se actualizan automáticamente. - Fácil de **\*\*añadir o quitar observadores\*\*** en tiempo de ejecución.

- Puede ser difícil de **depurar o seguir**, ya que los cambios se propagan automáticamente.
- Si hay muchos observadores, puede impactar en el **rendimiento**.
- El orden en que se notifican no está garantizado (salvo que lo controles explícitamente).

## 10.2 State

Permite que un objeto **modifique su comportamiento** cada vez que cambia su estado. **Parecerá** que **cambia la clase** de ese objeto. Se usa cuando:

- El comportamiento de un objeto **depende de su estado** y cambiar en **tiempo de ejecución**.
- Algunos métodos presentan estructuras **condicionales con múltiples ramas** formadas por bloques extensos de código.

### Participantes

Participante	Rol
<b>Sujeto (Subject)</b>	El objeto que tiene el estado que puede cambiar. Mantiene una lista de observadores y define métodos para añadir, eliminar y notificar observadores.
<b>Observador (Observer)</b>	Interfaz que deben implementar los objetos que quieren enterarse de los cambios. Contiene el método <code>actualizar()</code> .
<b>SujetoConcreto</b>	Implementación del sujeto, guarda el estado que interesa a los observadores y los notifica cuando cambia.
<b>ObservadorConcreto</b>	Implementación del observador. Se registra al sujeto y actualiza su estado cuando recibe una notificación.